

Arquitetura do Sistema de Eventos

Autor: Gustavo Ricardo Moreira Silva
Bruno Silva Lima
Rafyc Lisboa Maia Santos
Carlos Patrick de Aguiar Lima Costa
Ryan Filipe Sousa Motta Santana

Agenda

O objetivo dessa apresentação é descrever a arquitetura de software e a estrutura de dados de um Sistema de Gestão de Eventos.

1. Contexto

Evolução da prova de conceito inicial para uma arquitetura escalável e resiliente.

2. DTO em camadas

Especificação do fluxo de entrada de dados e contratos de interface do sistema.

3. Controller

Ponto de entrada da aplicação, responsável por orquestrar as requisições e gerenciar os recursos de eventos..

4. Builder

Implementação do padrão criacional para construção de objetos complexos de eventos de forma fluida.

5. DTO

Representação do estado atual do evento para transporte entre diferentes camadas do sistema.

6. Mappers

Camada de tradução responsável por converter objetos de domínio em representações de transferência (e vice-versa).

7. Diagramas UML

Modelagem estática do sistema detalhando as relações e dependências entre as classes.

8. Diagrama de Atividade

Mapeamento visual do fluxo de trabalho e da lógica de execução dos processos de negócio..

Contexto Inicial (Dendê Softhouse)



A Dendê Softhouse desenvolve o **Dendê Eventos**, uma plataforma para gestão de eventos culturais e acadêmicos. Na primeira etapa, modelamos o sistema com um Diagrama de Classes e implementamos os primeiros endpoints em Java puro e POO, sem frameworks externos. Já na segunda etapa, focamos na excelência técnica através da refatoração, aplicando padrões como **Builder**, **DTOs**, **Mappers** e **Controllers** para tornar o código mais modular e seguro. Além disso, atualizamos a modelagem com um Diagrama de Atividades para um endpoint específico.

Fizemos a implementação dos primeiros endpoints da API utilizando Java puro. Agora é a refatoração do código através da aplicação de padrões de projeto como Builder, DTOs e Mappers.

DTO em camadas (Controller, Service e Repositor)

- **Camada Controller:** É a porta de entrada. O Controller recebe um **Request DTO** (dados vindos do cliente) e, após o processamento, retorna um **Response DTO**. Isso evita expor diretamente as entidades do banco de dados e garante que apenas as informações necessárias sejam trafegadas.
- **Camada Service:** É onde a "mágica" da conversão acontece. O Service recebe o DTO do Controller, utiliza um **Mapper** para transformá-lo em uma **Entidade** (como a classe **Evento**) e executa as regras de negócio.
- **Camada Repository:** Lida puramente com as **Entidades**. Ela persiste ou busca os objetos do modelo no "banco" (ou lista). O Repository não deve conhecer os DTOs; ele entrega a Entidade para o Service, que então a mapeia de volta para um DTO de resposta.

- **Ponto de Entrada:** Funcionam como a porta de entrada da API, gerenciando as requisições HTTP enviadas para cada classe do modelo.
- **Isolamento de Dados:** Utilizam os **DTOs** para receber e retornar informações, garantindo que os dados sensíveis do modelo interno não sejam expostos diretamente nos endpoints.
- **Orquestração:** Recebem os dados da requisição, acionam os **Mappers** para conversão e direcionam para a lógica de negócio, retornando a resposta adequada ao cliente.
- **Organização por Recurso:** São estruturados de forma modular, onde cada Controller foca em uma entidade específica (como Usuário ou Evento) baseando-se nas rotas do repositório.

Controller



```
@Controller
@RequestMapping
public class EventoController {

    private final EventoService eventoService = new EventoService();

    // 1. Rota (Endpoint) e Verbo HTTP (POST)
    @PostMapping(path = "/organizadores/{email}/eventos")
    public ResponseEntity<EventoDTO> cadastrar(
        // 2. Extrai variáveis da URL e do Corpo da Requisição
        @PathVariable(parameter = "email") String email,
        @RequestBody EventoRequestDTO request
    ) {
        try {
            // 3. Delega a regra de negócio para a camada de Service
            Evento novoEvento = eventoService.cadastrarEvento(email, request);

            // 4. Converte para DTO e retorna com Status HTTP 201 (Created)
            return ResponseEntity.status(201, EventoMapper.toDTO(novoEvento));

        } catch (IllegalArgumentException e) {
            // 5. Tratamento de erro padronizado (400 Bad Request)
            return ResponseEntity.status(400, null);
        }
    }
}
```

Padrão Builder (Criação de Objetos)

- **O que resolve:** Evita "construtores gigantes" e a confusão na ordem de parâmetros ao criar a classe **Evento**, que possui muitos atributos.
- **Legibilidade:** Elimina a necessidade de passar diversos valores nulos no construtor.
- **Flexibilidade:** Permite configurar apenas os campos necessários de forma encadeada.
- **Nomenclatura:** Os métodos do Builder devem ter o mesmo nome dos atributos para facilitar a identificação do que está sendo configurado.

Padrão Bidirecional (Relacionamentos)

- **O que resolve:** Garante que dois objetos "se conheçam" simultaneamente no modelo conceitual.
- **Navegação:** Facilita o acesso aos dados;
- **Eficiência:** Evita buscas complexas em listas;



1.

```
public class EventoBuilder {  
  
    private int id;  
    private String nome;  
    private boolean ativo = false;  
  
    // 1. Construtor privado  
    private EventoBuilder() {}  
  
    // 2. Método de inicialização estático  
    public static EventoBuilder builder() {  
        return new EventoBuilder();  
    }  
  
    // 3. Métodos fluentes  
    public EventoBuilder id(int id) {  
        this.id = id;  
        return this;  
    }  
}
```

2.

```
public EventoBuilder nome(String nome) {  
    this.nome = nome;  
    return this;  
}  
  
public EventoBuilder ativo(boolean ativo) {  
    this.ativo = ativo;  
    return this;  
}  
  
// 4. Método build de construção final  
public Evento build() {  
    Evento evento = new Evento();  
    evento.setId(this.id);  
    evento.setName(this.nome);  
    evento.setAtivo(this.ativo);  
    return evento;  
}  
}
```


O que é?

- É um objeto simples em Java que agrupa dados para uma finalidade específica.
- No projeto, serve para separar o modelo interno da aplicação da interface externa da API.

Para que serve?

- **Segurança:** Evita a exposição direta de entidades do banco e dados sensíveis ao cliente.
- **Eficiência:** Otimiza o tráfego ao enviar apenas os campos necessários para cada funcionalidade ou tela.
- **Flexibilidade:** Permite alterar a estrutura interna do banco sem quebrar o contrato de comunicação com o usuário.

Quando usar?

- **Requisições (Requests):** Para capturar dados de cadastros e atualizações enviados pelo usuário.
- **Respostas (Responses):** Para formatar e filtrar o que será exibido no feed ou perfil do usuário.
- **Entre Camadas:** Para transferir dados do **Controller** para o **Service**, garantindo o isolamento das regras de negócio.

```
public class EventoRequestDTO {  
    private String nome;  
    private String paginaWeb;  
    private String descricao;  
    private LocalDateTime dataInicio;  
    private LocalDateTime dataFim;  
    private TipoEvento tipoEvento;  
    private ModalidadeEvento modalidade;  
    private Double precoUnitarioIngresso;  
    private Double taxaCancelamentoIngresso;  
    private int capacidadeMaxima;  
    private String local;  
    private int eventoPrincipalId;
```

Mappers



- **O que é:** Responsável por "traduzir" objetos, convertendo Entidades de negócio (Model) em objetos de transferência de dados (DTOs) e vice-versa.
- **Por que utilizamos no Dendê Eventos?**
 - **Segurança:** Protege o modelo interno.
 - **Desacoplamento:** Separa o formato dos dados que a API recebe/envia do formato que o banco de dados armazena.
 - **Encapsulamento de Lógica:** O Mapper centraliza lógicas de apresentação.

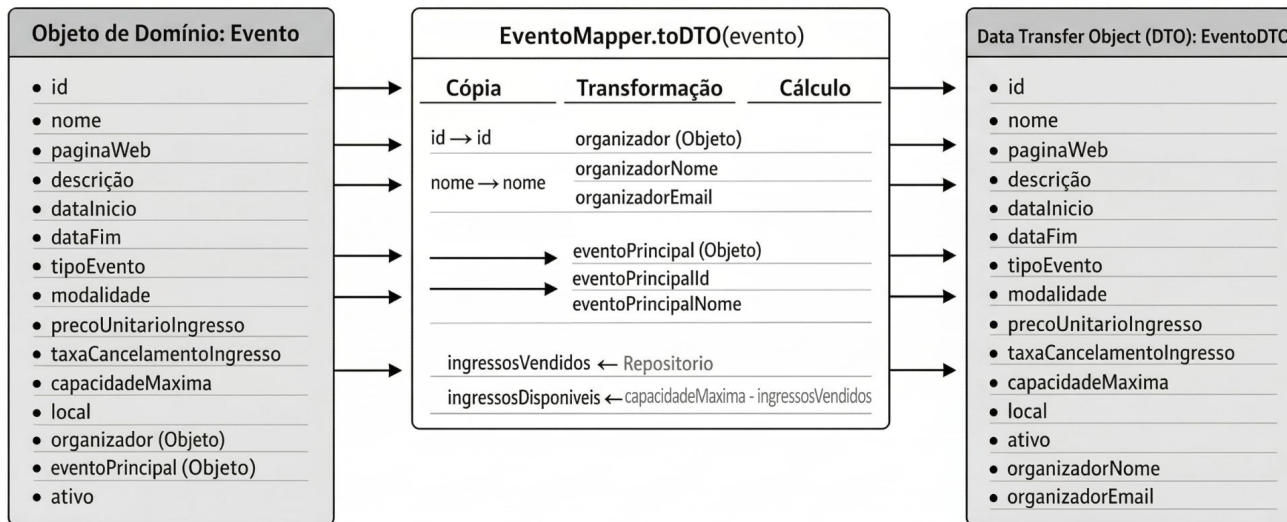


Diagrama UML (EVENTO)

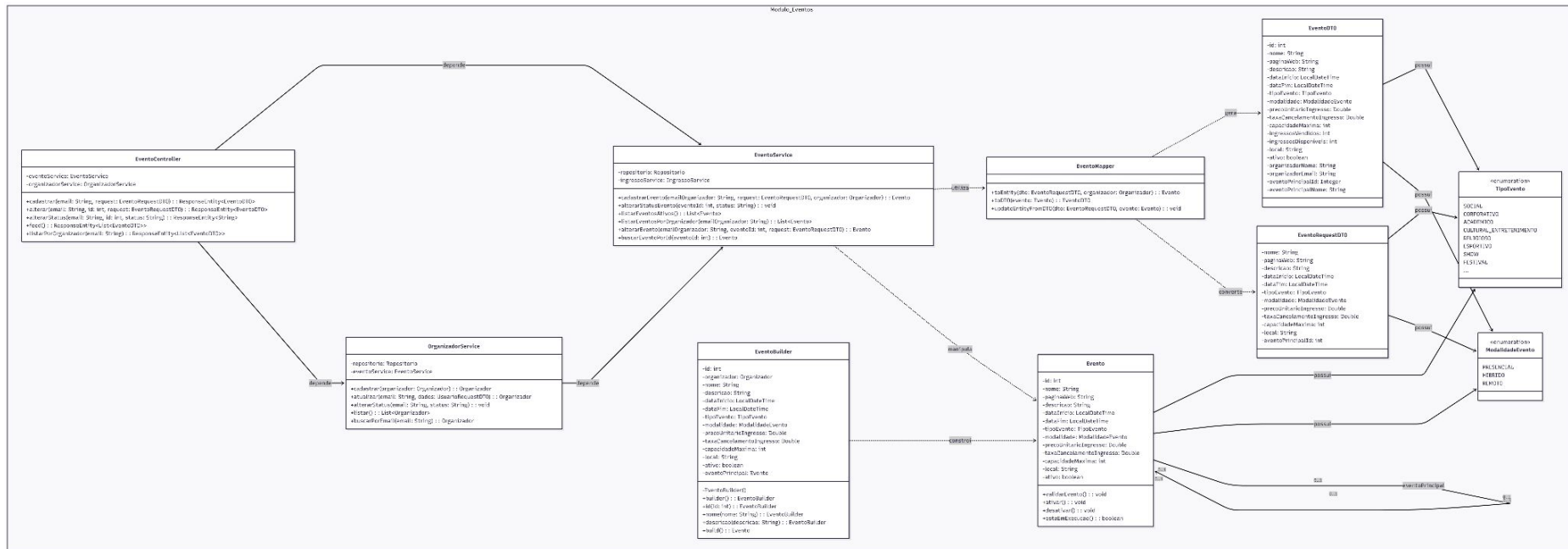


Diagrama UML(INGRESSO)

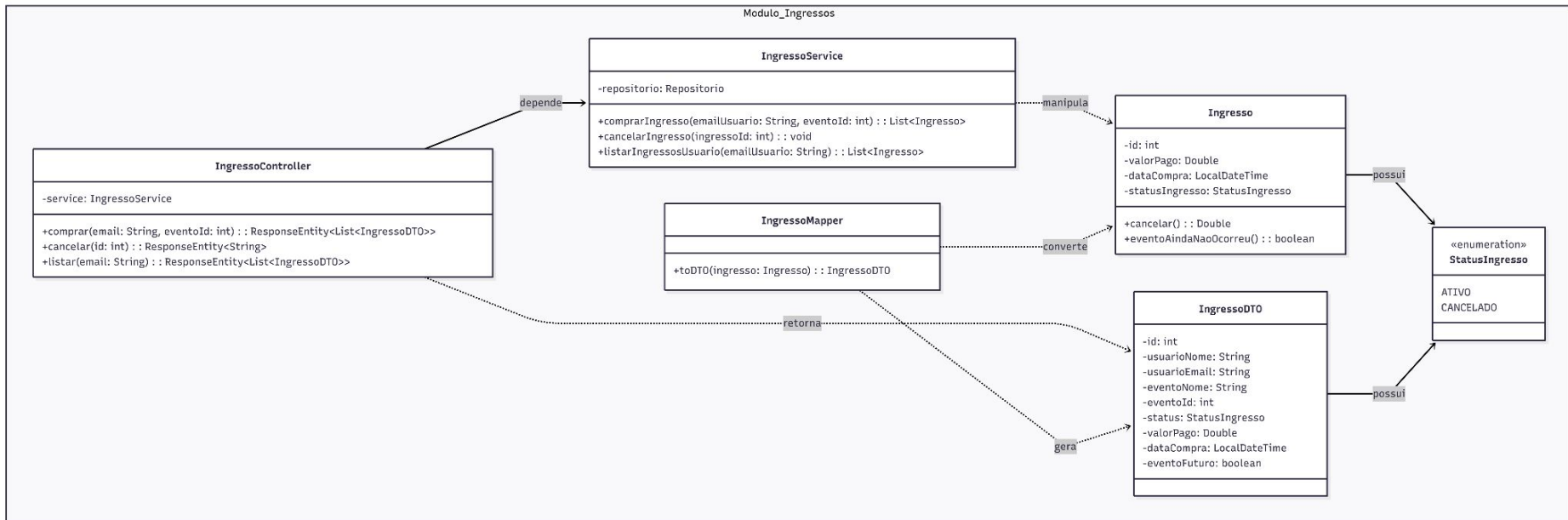


Diagrama UML(USUÁRIO)

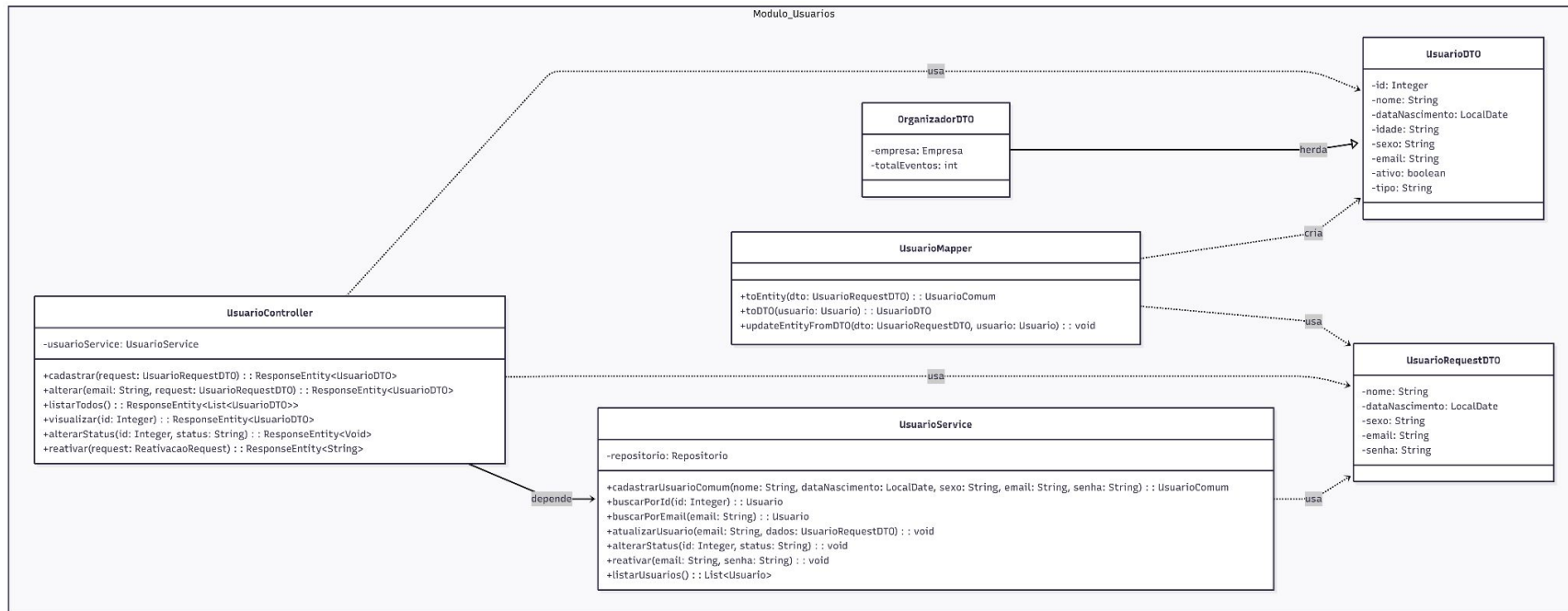


Diagrama UML (REPOSITORIO)

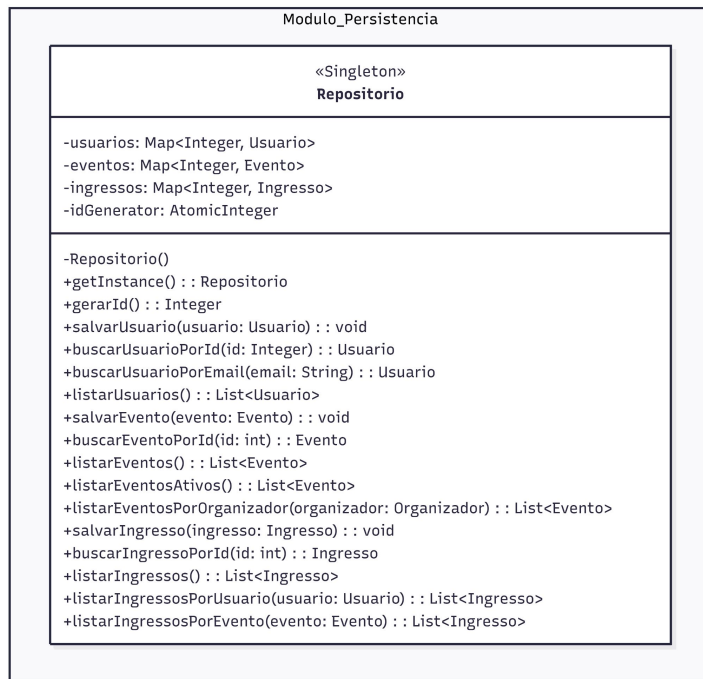
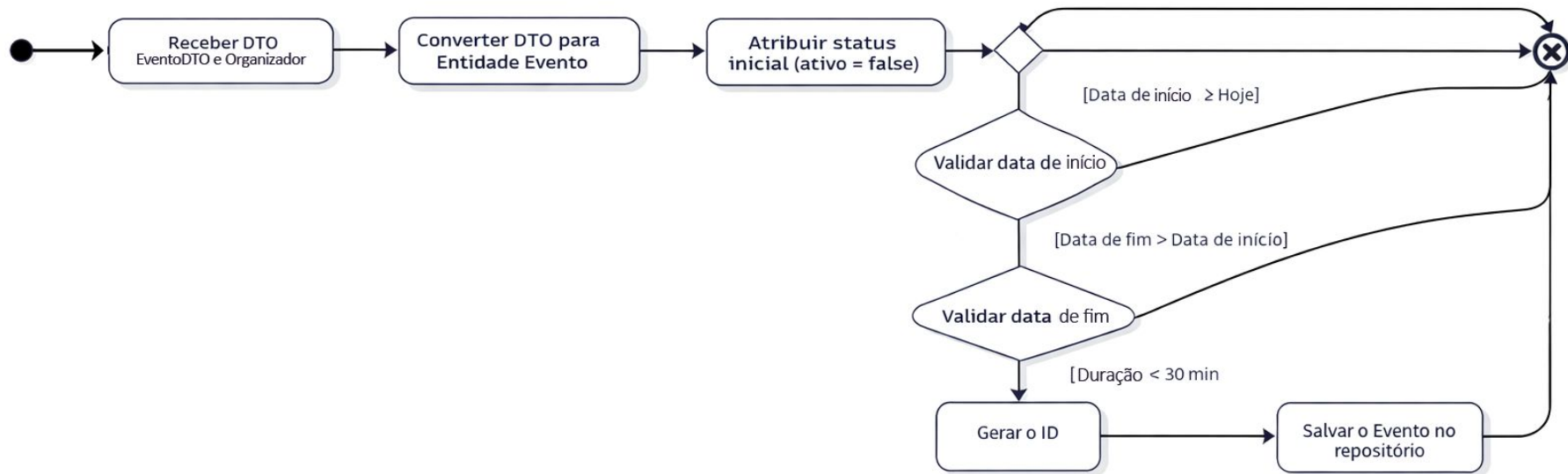


Diagrama de Atividade



Considerações finais



A Dendê Softhouse desenvolve o app Dendê Eventos e está refatorando o sistema em Java para melhorar **modularidade e segurança**. Foram usados **Builder**, **DTOs** e **Mappers** para organizar e proteger os dados, além de **Controllers** e diagramas de atividades para garantir uma arquitetura robusta e preparada para futuras evoluções.

Referências



Oracle Corporation. Java Platform, Standard Edition Documentation. Disponível em:
<https://docs.oracle.com/javase/>

Object Management Group. Unified Modeling Language (UML) Specification. Disponível em:
<https://www.omg.org/spec/UML/>

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

FOWLER, M. UML Distilled: A Brief Guide to the Standard Object Modeling Language. 3. ed. Addison-Wesley, 2003.